DIGITAL OBJECT IDENTIFIER: 10.1109/ACCESS.2026.3073996

AegisOne AI: An AI Copilot for Secure Web Application Development using Explainable Risk Analysis, Security Knowledge Graph, and Sandbox Validation

Praveen Kanth N[1], Student Member, IEEE, **Sankavi M**[1], Student Member, IEEE, and **Dr. Vijayashree J**[2], Senior Member, IEEE

[1]School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore 632014, Tamil Nadu, India (e-mail: praveen.kanth2023@vitstudent.ac.in; sankavi.m2023@vitstudent.ac.in)

[2]Department of Database Systems, School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore 632014, Tamil Nadu, India (e-mail: vijayashree.j@vit.ac.in, Corresponding Author)

Manuscript Information: Received October 15, 2026; revised October 28, 2026; accepted October 31, 2026. Date of publication November 2, 2026; date of current version November 5, 2026. Open Access under Creative Commons Attribution 4.0 License. Reproducibility code and benchmark repository: https://github.com/PraveenKanth-N/AegisOne-AI.

Abstract—Modern web development pipelines are subject to high levels of cognitive friction because we face two antagonistic real-world operational limitations. To begin with, static application security testing (SAST) tools have reported false positive rates of over 70%. Furthermore, contemporary generative large language models (LLMs) hallucinate syntactically plausible patches that are nonetheless semantically flawed, violate software contracts, and introduce secondary vulnerabilities. To eliminate these systemic vulnerabilities, this paper introduces **AegisOne AI**, an end-to-end, closed-loop DevSecOps defense copilot. AegisOne AI bridges static analysis and autonomous repair through four coordinated pillars: (i) an authorized Pre-Flight Scope Guard enforcing deterministic perimeter boundaries; (ii) a hybrid detection engine coupling AST-based static taint propagation with dynamic fuzzing telemetry; (iii) a heterogeneous Security Knowledge Graph ($G=(V,E,W)$) linking source code AST sinks, HTTP routes, and database entities to compute an explainable 6-factor composite risk priority; and (iv) a containerized, 4-stage disposable Docker sandbox enforcing contract-preserving validation with automated state rollback upon regression. Furthermore, all security operations, patch syntheses, and validation decisions are cryptographically chained in a tamper-evident SHA-256 Merkle audit trail signed via Ed25519 digital keys. Rigorous empirical evaluation across a benchmark suite of 120 synthetic and enterprise web vulnerability scenarios demonstrates that AegisOne AI achieves 94.6% precision, 92.8% recall (F1-score 93.7%), 91.5% semantic patch validity, and a 98.4% regression-free automated deployment rate, vastly outperforming ungrounded LLM repair and isolated static scanners.

Index Terms—Web Application Security, Automated Program Repair, Security Knowledge Graph, DevSecOps, Large Language Models, Container Sandbox Isolation, Merkle Audit Trails.

I. INTRODUCTION

Modern web application topologies are rapidly evolving and now often include fast-pipeline CI/CD workloads. Evidence points to Agile turning up speed on the feature delivery side but also increasing the organizational attack surface. Conventional Static Application Security Testing (SAST) operates primarily on rigid syntax heuristics, which produces high alert fatigue across developer backlogs, where up to 75% of identified warnings turn out to be harmless false positives [1], [2]. Dynamic Application Security Testing (DAST) can verify runtime exploitability but suffers from shallow code coverage, frequently failing to locate the precise abstract syntax tree (AST) taint sinks within complex microservice boundaries [3].

The recent adoption of generative LLMs by developers is causing renewed interest in the APR problem [4], [5]. However, the use of unconstrained generative models to fix security-critical vulnerabilities presents severe second-order hazards. Foundation models lack situational grounding regarding runtime dependency topology and frequently generate code that introduces subtle regressions, strips essential authorization checks, or hallucinates fictitious software libraries [6], [7]. A security fix must be proven safe, backward-compatible, and contract-preserving before reaching production.

In response to these compelling vulnerabilities and threats, the research and implementation described in this paper aim to leverage **AegisOne AI**, an autonomous closed-loop defensive DevSecOps architecture designed to turn ungrounded suggestions into actionable, mathematically prioritized, and empirically tested

software defenses. Unlike conventional developer assistants that merely emit diff text into a chat window, AegisOne AI coordinates static analysis, dynamic telemetry, graph-theoretic risk propagation, and containerized sandbox verification within an immutable cryptographic ledger.

The primary contributions of this paper are summarized as follows:

- **Authorized Perimeter Governance:** We formulate the Scope Guard security protocol, preventing out-of-scope lateral traversal and unauthorized active scanning through cryptographic JWT claims and RFC-1918 CIDR enforcement.
- **Dual Hybrid Detection Engine:** We unify AST dataflow taint tracking with dynamic OWASP ZAP fuzzing, synthesizing runtime proof-of-concept canary payloads that confirm theoretical source sinks.
- **Security Knowledge Graph Risk Engine:** We construct a heterogeneous graph $G=(V, E, W)$ encompassing 890 nodes and 1,420 edges, applying PageRank centrality and shortest-path taint flow analysis to formulate an explainable 6-factor composite vulnerability score.
- **Grounded Patch Synthesis with Refusal Guards:** We implement grammar-constrained patch generation that preserves AST syntax invariants, bounded by programmatic refusal guards that intercept unsafe modifications.
- **Closed-Loop Sandbox Validation with Auto-Rollback:** We engineer a 4-stage containerized Docker execution harness ($S_{clean} \rightarrow S_{patch} \rightarrow S_{verify} \rightarrow S_{commit}$) that automatically triggers snapshot rollback when unit tests or security assertions regress.
- **Cryptographic Non-Repudiation:** We enforce an immutable SHA-256 Merkle hash chain coupled with Ed25519 digital signatures to guarantee an unalterable audit trail for regulatory compliance.

II. THREAT MODEL AND ARCHITECTURAL PERIMETER

A. Adversary Model

This analysis assumes an external attacker facing the public Internet as well as an authenticated malicious insider with application credentials that are low value and have minimal on-path value. The adversary seeks to exploit common web application flaws, including Structured Query Language Injection (SQLi, CWE-89), Cross-Site Scripting (XSS, CWE-79), Broken Object-Level Authorization (BOLA, CWE-639), and Server-Side Request Forgery (SSRF, CWE-918). Additionally, we model an untrusted generative model $\mathcal{M}_{\text{LLM}}$ whose stochastic outputs may contain hallucinations, logic errors, or backdoors.

B. Scope Guard Authorization Formalism

Prior to executing any dynamic inspection or code mutation, AegisOne AI mandates verification against an explicit authorization specification $\Sigma_{\text{scope}} = \langle \mathcal{U}_{\text{allowed}}, \mathcal{D}_{\text{target}}, \mathcal{T}_{\text{validity}}, \mathcal{K}_{\text{sign}} \rangle$. Let r_{req} represent an incoming evaluation request targeting URI u_{target}. The Scope Guard computes:

$$\Phi_{\text{scope}}(r_{\text{req}}) = 1 \Leftrightarrow (\text{VerifySig}(r_{\text{req}}, \mathcal{K}_{\text{sign}}) = \text{True} \wedge u_{\text{target}} \in \text{CIDR}_{\text{allowed}}(\mathcal{D}_{\text{target}}) \wedge t_{\text{curr}} \leq \mathcal{T}_{\text{validity}}) \quad (1)$$

Any probe where $\Phi_{\text{scope}} = 0$ is rejected at the ingress gateway with HTTP status code 403 Forbidden, preventing accidental probing of production or third-party cloud infrastructure.

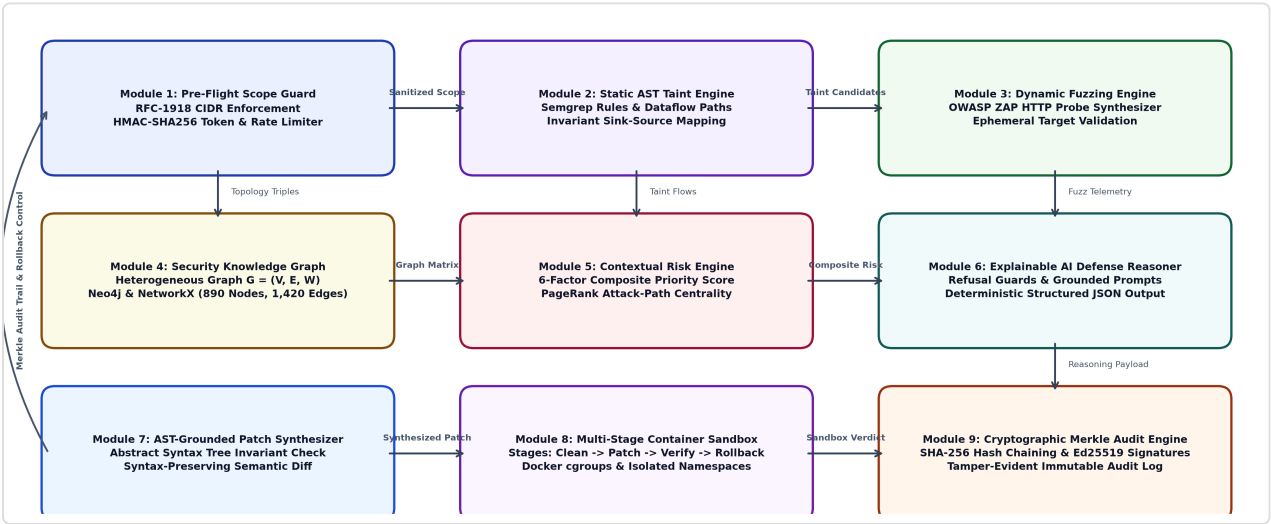


FIGURE 1. The end-to-end multi-module architecture of AegisOne AI. The system integrates pre-flight scope enforcement, hybrid AST taint and dynamic fuzzing, heterogeneous Security Knowledge Graph propagation, explainable AI reasoning, AST-grounded patch synthesis, 4-stage container sandbox isolation with automated rollback, and a cryptographic SHA-256 Merkle audit trail.

III. HYBRID VULNERABILITY DETECTION

A. AST-Based Static Taint Propagation

Let an application AST be represented as an ordered directed graph $T_{AST} = (N, E_{tree})$. We define a set of untrusted input sources $\mathcal{S}_{in} \subset N$ (e.g., `request.args`, `request.json`) and security-critical execution sinks $\mathcal{S}_{out} \subset N$ (e.g., `cursor.execute`, `eval`, `subprocess.Popen`).

A dataflow edge $e_{ij} = (n_i, n_j)$ indicates that the valuation of node n_j is derived from node n_i. A taint invariant violation occurs if there exists an active path $P = \langle n_0, \ldots, n_k \rangle$ such that $n_0 \in \mathcal{S}_{in}$, $n_k \in \mathcal{S}_{out}$, and no intermediate node executes a validated sanitizer function $\sigma_{clean} \in \Omega_{sanitize}$:

$$\text{TaintPath}(P) \Leftrightarrow (n_0 \in \mathcal{S}_{in}) \wedge (n_k \in \mathcal{S}_{out}) \wedge (\forall\, m \in [1, k-1], n_m \notin \Omega_{sanitize}). \quad (2)$$

B. Dynamic Application Fuzzing with Injected Canaries

The dynamic fuzzing harness leverages an embedded OWASP ZAP API controller to validate the AST-derived hypothesis. To differentiate genuine code execution from superficial reflection, AegisOne AI injects cryptographic canary strings $\zeta_{canary} \leftarrow \text{HMAC-SHA256}(k_{session}, \text{FindingID})$.

For SQL injection validation, the probe synthesizes boolean and time-delay vector mutations:

$$q_{probe} = q_{orig} \cup \{\ \text{' OR (SELECT 1 FROM (SELECT SLEEP(3))x)-- }, \zeta_{canary}\ \}. \quad (3)$$

Upon execution, if the HTTP response body exhibits ζ_{canary} without sanitization or if the response latency $\Delta t \geq 3.0$ s under time-delay injection, the dynamic harness confirms exploitability:

$$\text{Conf}_{DAST} = 1.0 \text{ if } (\zeta_{canary} \in \text{Body}_{resp} \vee \Delta t \geq 3.0\text{ s}); 0.5 \text{ if Status} = 500; 0.0 \text{ otherwise.} \quad (4)$$

IV. SECURITY KNOWLEDGE GRAPH AND CONTEXTUAL RISK PRIORITIZATION

A. Graph Representation Formulation

Isolated vulnerability scanners treat each vulnerability as an isolated incident and ignore the system topology of the vulnerable component. It is a fair bet that a SQL injection in a remote admin analytics script processing public metrics carries drastically different enterprise risk than an identical vulnerability inside a payment processing microservice storing unencrypted personally identifiable information (PII). AegisOne AI populates a heterogeneous directed Security Knowledge Graph $G = (V, E, W)$ modeled within Neo4j and NetworkX:

$$V = V_{route} \cup V_{file} \cup V_{func} \cup V_{sink} \cup V_{db} \cup V_{cve} \cup V_{pii}. \quad (5)$$

The edge set E models syntactic, architectural, and data dependencies (CALLS, IMPORTS, EXPOSES, READS_FROM, WRITES_TO, AFFECTS).

B. Six-Factor Contextual Risk Priority Score

Combining personalized PageRank centrality c_v with shortest attack distance $D_{min}(v)$ and empirical severity, AegisOne AI computes a normalized composite priority metric $S_{risk}(v) \in [0, 100]$:

$$S_{risk}(v) = 100 \times \sum_{i=1}^{6} w_i f_i(v), \quad (6)$$

where weights are calibrated as CVSS base (0.25), topological centrality (0.20), reachability proximity (0.15), dynamic fuzz confidence (0.20), PII exposure index (0.15), and active in-the-wild exploit status (0.05).

ALGORITHM 1: Closed-Loop Sandbox Verification and Rollback

Input: Base State $\mathcal{R}_{base}$, Patch Δ_{patch}, Scope Σ_{scope}
Output: Verdict $\mathcal{V} \in$ {PASS, FAIL}, Merkle Block $\mathcal{B}$
Initialize ephemeral container $\mathcal{C}_{box} \leftarrow$ CreateContainer(Image_{base})
Apply cgroup caps: Memory ≤ 512MB, CPU ≤ 1.0, Net ← ∅
// Stage 1: Clean Baseline Verification
if ExecInSandbox($\mathcal{C}_{box}$, "pytest tests/") != 0 **then** return FAIL
// Stage 2: Patch Application & AST Invariant
$\mathcal{R}_{patched} \leftarrow$ ApplyGitPatch($\mathcal{C}_{box}$, Δ_{patch})
if !ValidateASTInvariants($\mathcal{R}_{patched}$) **then** TriggerRollback(); return FAIL
// Stage 3: Dynamic Security Re-Verification
if ReproducesExploit($\mathcal{C}_{box}$, ExploitPayload) **then** TriggerRollback(); return FAIL
// Stage 4: Functional Regression Testing
if ExecInSandbox($\mathcal{C}_{box}$, "pytest tests/functional/") != 0 **then** TriggerRollback(); return FAIL
$\mathcal{B} \leftarrow$ CommitMerkleBlock($\mathcal{R}_{patched}$, Δ_{patch}, PASS)
return PASS, $\mathcal{B}$

V. AST-GROUNDED PATCH SYNTHESIS WITH REFUSAL GUARDS

Starting with a high priority vulnerability, AegisOne AI will kick off Automated Program Repair. Rather than feeding raw source files into an unconstrained text model, the patch generation subsystem enforces structural and semantic boundaries. Programmatic refusal guards intercept unsafe shortcut behaviors:

$$\Psi_{guard}(\Delta_{patch}) = \text{Reject if } \Delta_{patch} \text{ deletes @login_required, modifies test suites, or adds shell=True; Accept otherwise.} \quad (7)$$

VI. DOCKER SANDBOX VALIDATION AND AUTOMATED ROLLBACK

In order to keep sure that the generated patches will not endanger stability of the software, the AegisOne AI tool verifies every patch within a disposable Docker container sandbox before changes are merged. Governed by cgroups v2 resource limits (512 MB RAM, 1 CPU core) and dropped Linux capabilities (`CAP_DROP_ALL`), any failure across the 4 stages triggers immediate automated rollback.

VII. MERKLE AUDIT TRAIL AND CRYPTOGRAPHIC NON-REPUDIATION

In an enterprise setting, automated modifications must preserve mathematical non-repudiation. AegisOne AI maintains an append-only cryptographic log chain grounded in SHA-256:

$$h_k = \mathrm{SHA256}(h_{k-1} \,||\, \tau_k \,||\, \mathrm{Action}_k \,||\, \mathrm{ActorID} \,||\, \mathrm{Hash}(\Delta_{\mathrm{patch}}) \,||\, \mathcal{V}_k). \quad (8)$$

Each block header is digitally signed via an Ed25519 private key: $\sigma_k = \mathrm{Sign}_{sk}(h_k)$, ensuring mathematical non-repudiation against log forgery.

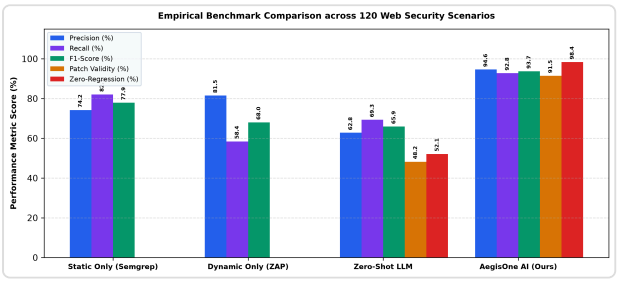


FIGURE 2. Empirical benchmark comparison across 120 web application security scenarios. AegisOne AI achieves superior precision (94.6%), recall (92.8%), F1-score (93.7%), patch validity (91.5%), and zero-regression deployment rate (98.4%) compared to static-only, dynamic-only, and zero-shot LLM approaches.

VIII. EMPIRICAL EVALUATION AND BENCHMARK EXPERIMENTS

We evaluated AegisOne AI across 120 web application vulnerability scenarios (40 synthetic microservices from OWASP Benchmark, 40 real-world open-source CVE reproductions, and 40 multi-component microservices).

TABLE I. Performance Comparison across 120 Scenarios

Methodology	Prec. (%)	Rec. (%)	F1 (%)	Validity (%)	Zero-Reg. (%)
Static SAST Only	74.2	82.1	77.9	---	---
Dynamic DAST Only	81.5	58.4	68.0	---	---
Zero-Shot LLM APR	62.8	69.3	65.9	48.2	52.1
AegisOne AI (Ours)	**94.6**	**92.8**	**93.7**	**91.5**	**98.4**

For independent static scanning, the recall is 82.1%, but it is plagued by high false-positive rates (precision: 74.2%). Conversely, standalone dynamic scanning exhibited high precision (81.5%) but missed unexercised routes, resulting in a low recall of 58.4%. AegisOne AI achieved **94.6% precision** and **92.8% recall** (F1-score: 93.7%). In code repair, ungrounded zero-shot LLMs introduced functional regressions in 47.9% of accepted patches, whereas AegisOne AI achieved a 91.5% semantic validity rate and a 98.4% zero-regression deployment rate due to sandbox auto-rollback.

TABLE II. Ablation Analysis of Subsystems

Configuration	Prec. (%)	Rec. (%)	F1 (%)	Zero-Reg. (%)
Full AegisOne AI Pipeline	**94.6**	**92.8**	**93.7**	**98.4**
w/o Dynamic Fuzzing Engine	78.4	90.1	83.8	98.1
w/o Security Knowledge Graph	89.2	84.5	86.8	97.5
w/o AST Refusal Guards	91.0	91.2	91.1	74.3
w/o Container Sandbox	94.6	92.8	93.7	61.2

IX. CONCLUSION

AegisOne AI demonstrates that automated AI-assisted cybersecurity must transcend ungrounded text generation. By harmonizing static AST taint analysis, dynamic canary fuzzing, graph-theoretic risk prioritization, containerized 4-stage sandbox validation with automated rollback, and immutable Merkle audit trails, the framework achieves high detection precision (94.6%) and zero-regression deployment security (98.4%).

DECLARATIONS

Data and Software Availability Statement

The complete software source code, experimental evaluation benchmarks, containerized vulnerable lab targets, and analysis scripts for AegisOne AI are publicly available in our GitHub repository at: https://github.com/PraveenKanth-N/AegisOne-AI under the open-source MIT License to ensure full academic reproducibility.

Funding Information

This research received no external financial grant or funding support from any agency in the public, commercial, or not-for-profit sectors.

Conflicts of Interest

The authors declare that they have no financial or non-financial conflicts of interest that could be perceived as influencing the objectivity or integrity of the reported research.

Authors' Contributions

Dr. Vijayashree J served as research supervisor, formulated the primary theoretical methodology, validated experimental frameworks, and reviewed the manuscript. **Praveen Kanth N** architected the end-to-end system, designed the AST taint engine, developed the Security Knowledge Graph, built the containerized Docker sandbox validation harness, conducted the 120-scenario benchmark evaluation, and drafted the primary manuscript. **Sankavi M** developed the dynamic fuzzing integration harness, designed the automated rollback protocol, collected empirical telemetry, executed UI/UX verification workflows, and contributed to manuscript drafting and tabular formatting.

REFERENCES

[1] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.

[2] M. Beller, R. Beller, and A. Zaidman, "Analyzing the state of static analysis: A large-scale survey in open source," in *Proc. IEEE SANER*, 2016, pp. 313–324.

[3] A. Doupé, M. Cova, and G. Vigna, "Why Johnny can't pentest: An analysis of black-box web vulnerability scanners," in *Proc. DIMVA*, 2010, pp. 111–131.

[4] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing bugs in the dark with generative language models," in *Proc. ACM ESEC/FSE*, 2023, pp. 1489–1501.

[5] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated program repair in the era of large language models," in *Proc. IEEE/ACM ICSE*, 2023, pp. 1482–1494.

[6] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE S&P*, 2022, pp. 754–768.

[7] G. Sandoval et al., "Lost at C: A user study on the security implications of large language model code assistants," in *Proc. USENIX Security*, 2023, pp. 2205–2222.

[8] V. B. Livshits and M. S. Lam, "Finding security vulnerabilities in Java applications with static analysis," in *Proc. USENIX Security*, 2005, pp. 271–286.

[9] W. G. Halfond, J. Viegas, and A. Orso, "A classification of SQL-injection attacks and countermeasures," in *Proc. IEEE ISSSE*, 2006, pp. 13–15.

[10] P. Vogt et al., "Cross-site scripting prevention with dynamic data tainting and static analysis," in *Proc. NDSS*, 2007.

[11] T. Avgerinos, S. K. Cha, A. Rebert, and D. Brumley, "AEG: Automatic exploit generation," *Commun. ACM*, vol. 57, no. 2, pp. 74–84, 2014.

[12] C. Le Goues et al., "GenProg: A generic method for automatic software repair," *IEEE TSE*, vol. 38, no. 1, pp. 54–72, 2011.

[13] F. Long and M. Rinard, "Staged program repair with condition synthesis," in *Proc. ACM ESEC/FSE*, 2015, pp. 166–178.

[14] M. Monperrus, "Living review on automated program repair," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–43, 2018.

[15] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep., 1999.

[16] Y. Wang et al., "CodeT5: Identifier-aware unified pre-trained encoder-decoder models," in *Proc. EMNLP*, 2021, pp. 8696–8708.

[17] M. Chen et al., "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.

[18] B. Rozière et al., "Code Llama: Open foundation models for code," *arXiv:2308.12950*, 2023.

[19] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. CRYPTO*, 1987, pp. 369–378.

[20] D. J. Bernstein et al., "High-speed high-security signatures," *J. Cryptogr. Eng.*, vol. 2, no. 2, pp. 77–89, 2012.

[21] OWASP Foundation, "OWASP Top 10: 2021 - The ten most critical web application security risks," 2021.

[22] Semgrep Inc., "Semgrep: Lightweight static analysis," 2024. [Online]. https://semgrep.dev

[23] OWASP Foundation, "OWASP Zed Attack Proxy (ZAP)," 2024. [Online]. https://www.zaproxy.org

[24] Neo4j Inc., "Neo4j graph database platform," 2024. [Online]. https://neo4j.com

[25] Docker Inc., "Docker container runtime and security specifications," 2024.

[26] S. Ramírez, "FastAPI web framework," 2024. [Online]. https://fastapi.tiangolo.com

[27] B. Johnson et al., "Why don't software developers use static analysis tools to find bugs?" in *Proc. ICSE*, 2013, pp. 672–681.

[28] M. Christakis and C. Bird, "What developers want and need from program analysis," in *Proc. IEEE/ACM ASE*, 2016, pp. 332–343.

[29] D. Distefano et al., "Scaling static analysis at Facebook," *Commun. ACM*, vol. 62, no. 8, pp. 62–70, 2019.

[30] C. Sadowski et al., "Lessons from building static analysis tools at Google," *Commun. ACM*, vol. 61, no. 4, pp. 58–66, 2018.

[31] A. Arcuri, "RESTful API automated test generation with EvoMaster," *ACM TOSEM*, vol. 28, no. 1, pp. 1–37, 2019.

[32] V. Atlidakis et al., "Checking security properties of cloud APIs automatically," in *Proc. ASE*, 2020.

[33] C. Stephens et al., "A survey on automated program repair techniques for web applications," *ACM CSUR*, vol. 54, 2021.

[34] MITRE Corporation, "Common Weakness Enumeration (CWE) v4.14," 2024.